

STUDENT TO-DO LIST MANAGER

Project Report

Submitted by: Prachi Singh

Registration No.: 26BCE11448

Course/Class: B.Tech CS Core

College/School: VIT Bhopal (School-Scope)

Technology Used: Python, Tkinter, JSON, Git & GitHub

Academic Year: 2026–27

1. Introduction

Students often have several things to remember at the same time, such as assignments, practical work, project deadlines, tests and personal tasks. It is easy to forget a small task when everything is written in different places. The Student To-Do List Manager was made to provide one simple place where these tasks can be added and managed.

The application allows a student to enter a task, give it a priority and due date, and keep track of whether it has been completed. It also provides search, filtering and basic progress information. The main idea behind the project is to keep task management simple instead of making the application unnecessarily complicated.

2. Problem Statement

Managing academic work can become difficult when the number of assignments and deadlines increases. A student may remember a task but forget its due date, miss an important task, or find it difficult to see which work is still pending.

The project therefore focuses on building a small desktop application that can store tasks, update their status and make important tasks easier to find.

3. Objectives

The main objectives of the project are:

- To create a simple task-management application for students.
- To provide an easy way to add, update and delete tasks.
- To keep track of completed and pending work.
- To identify high-priority and overdue tasks.
- To provide search and filtering options.
- To save task information so it is available after restarting the application.

4. Proposed Solution

The proposed solution is a Python desktop application called Student To-Do List Manager. The user can enter the details of a task through the graphical interface. The application stores the information locally in a JSON file, so the tasks do not disappear when the program is closed.

Different parts of the program handle different jobs. Task management deals with adding, updating, completing and deleting tasks, while validation checks the entered information. Search and filtering help the user find particular tasks, and the analytics section gives a quick view of task progress.

5. Requirements

Hardware:

- Computer or laptop
- Minimum 4 GB RAM
- Keyboard and mouse

Software:

- Python 3
- Tkinter
- Git
- GitHub
- VS Code, PyCharm or another Python IDE

6. Functional Requirements

The application should provide the following functions:

- Add a task
- Update a task
- Delete a task
- Mark a task as completed
- Search for a task
- Filter tasks
- Set task priority
- Set a due date
- View basic progress and analytics
- Save and load tasks

7. Non-Functional Requirements

The application is designed with the following qualities in mind:

Usability: The interface should be straightforward so that a student can understand it without much instruction.

Reliability: Task information should be saved correctly and loaded again when the application is opened.

Performance: Common actions such as adding, searching and updating a task should respond quickly.

Maintainability: The program should be organised into separate parts so that changes can be made more easily.

Security: The project keeps the user's task data stored locally rather than sending it to an online service.

8. Design Diagrams

The report should include five diagrams to explain how the project is designed:

1. Use Case Diagram – shows the actions available to the student.
2. Workflow Diagram – shows the normal flow of the application from entering a task to saving or updating it.
3. Sequence Diagram – shows the interaction between the user, GUI, task manager and storage.
4. Class/Component Diagram – shows the main Python modules and their relationships.
5. ER Diagram – represents the structure of the task data stored in JSON.

Insert the actual diagrams below this section after creating them.

9. Design Decisions & Rationale

Python was chosen because it is familiar, readable and suitable for a student-level desktop project. It also provides the libraries needed for the application.

Tkinter was used for the graphical interface because it comes with Python and provides common GUI elements without requiring a large framework.

JSON was selected for storage because the data is small, easy to read and simple to handle from Python.

The program is divided into separate areas such as task management, validation, storage, search and analytics. This makes the code easier to understand and modify.

The application follows a single-user design. It is intended for an individual student, so features such as multi-user collaboration were kept outside the current scope.

10. Implementation Details

The Student To-Do List Manager is implemented in Python. The main components of the application are:

Task Model: Stores details such as task ID, title, description, due date, priority, category and status.

Task Manager: Handles operations such as adding, updating, completing and deleting tasks.

Validation: Checks the task information entered by the user.

Storage: Saves and loads task information using a JSON file.

Search: Helps the user find a particular task.

Analytics: Provides information such as completion rate and high-priority tasks.

GUI: Provides the interactive desktop interface using Tkinter.

11. Screenshots / Results

The following screenshots show the live Python Tkinter application in operation, demonstrating core features including Taskloading, data entry validation, search querying, priority filtering, and dynamic analytics

Figure 1- Main Application Interface & Initial Dashboard

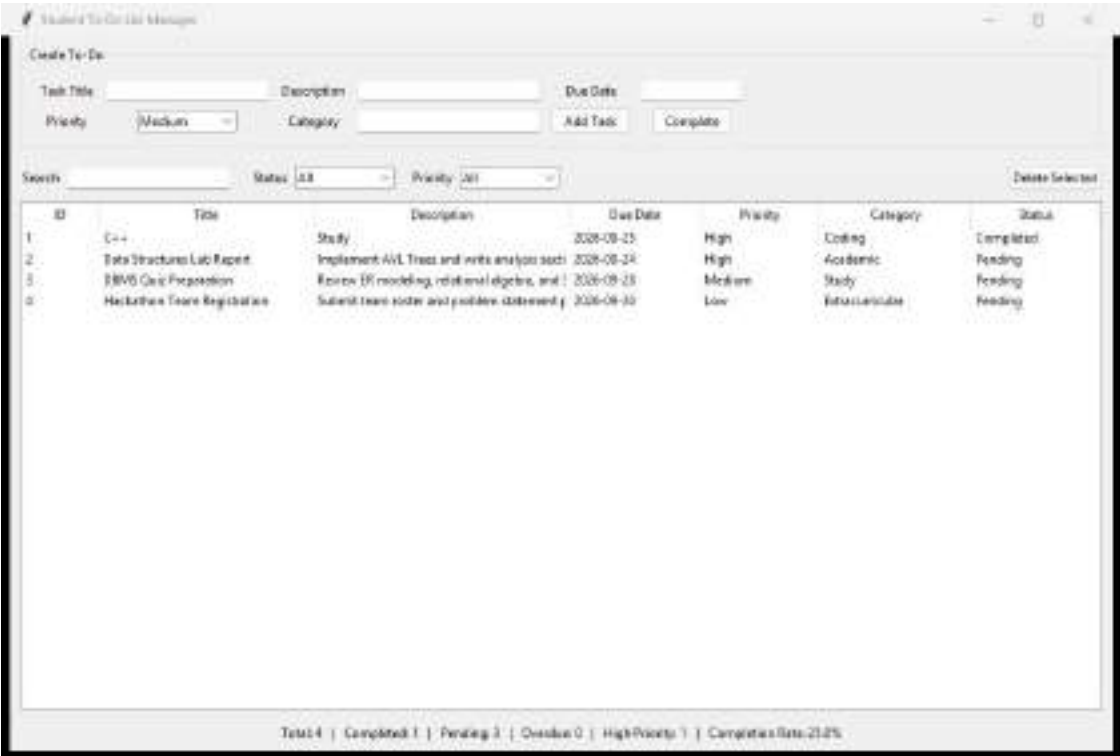


Figure 1: Shows the main window with all tasks loaded into the multi-column Treeview table and the live analytics status bar at the bottom.

Figure 2: Task Creation Form & Metadata Input

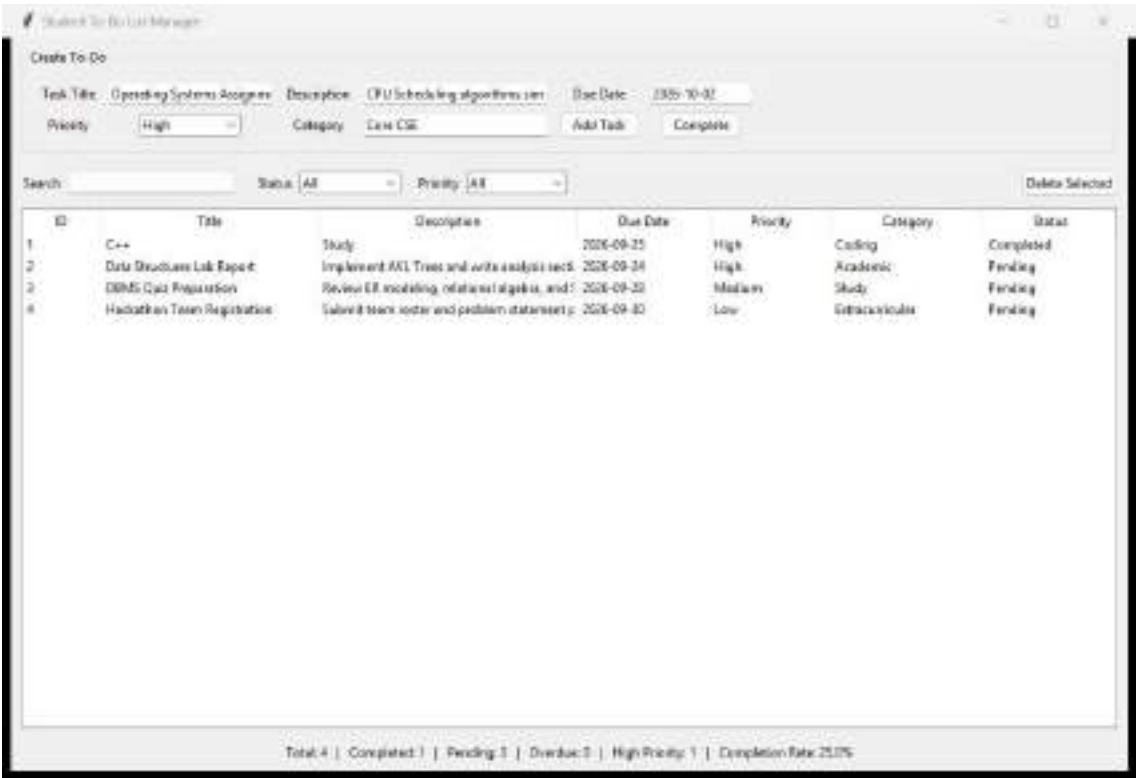


Figure 3: Real-Time Keyword Search Functionality

Create To-Do

Task Title

Description

Due Date

Priority

Medium

Category

Add Task

Complete

Search: Lab

Status: All

Priority: All

Delete Selected

ID	Title	Description	Due Date	Priority	Category	Status
2	Data Structures Lab Report	Implement AVL Trees and write analysis secti	2028-09-28	High	Academic	Pending

Total: 4 | Completed: 1 | Pending: 3 | Overdue: 0 | High Priority: 1 | Completion Rate: 25%

Figure 3: Live query matching the substring 'Lab', filtering the table dynamically down to the matching task record.

Figure 4: Multi-Criteria Priority Filtering

Create To-Do

Task Title

Description

Due Date

Priority

Medium

Category

Add Task

Complete

Search:

Status: All

Priority: High

Delete Selected

ID	Title	Description	Due Date	Priority	Category	Status
1	C++	Study	2028-09-25	High	Coding	Completed
2	Data Structures Lab Report	Implement AVL Trees and write analysis secti	2028-09-28	High	Academic	Pending

Total: 4 | Completed: 1 | Pending: 3 | Overdue: 0 | High Priority: 1 | Completion Rate: 25%

Figure 4: Priority dropdown set to 'High', isolating urgent pending and completed items for immediate academic focus.

Figure 5: Task Completion & Dynamic

Testing was carried out to check whether the main features of the application behaved as expected. The following test cases can be used as a record of testing. The final Pass/Fail status should be filled in only after the corresponding feature has actually been tested.

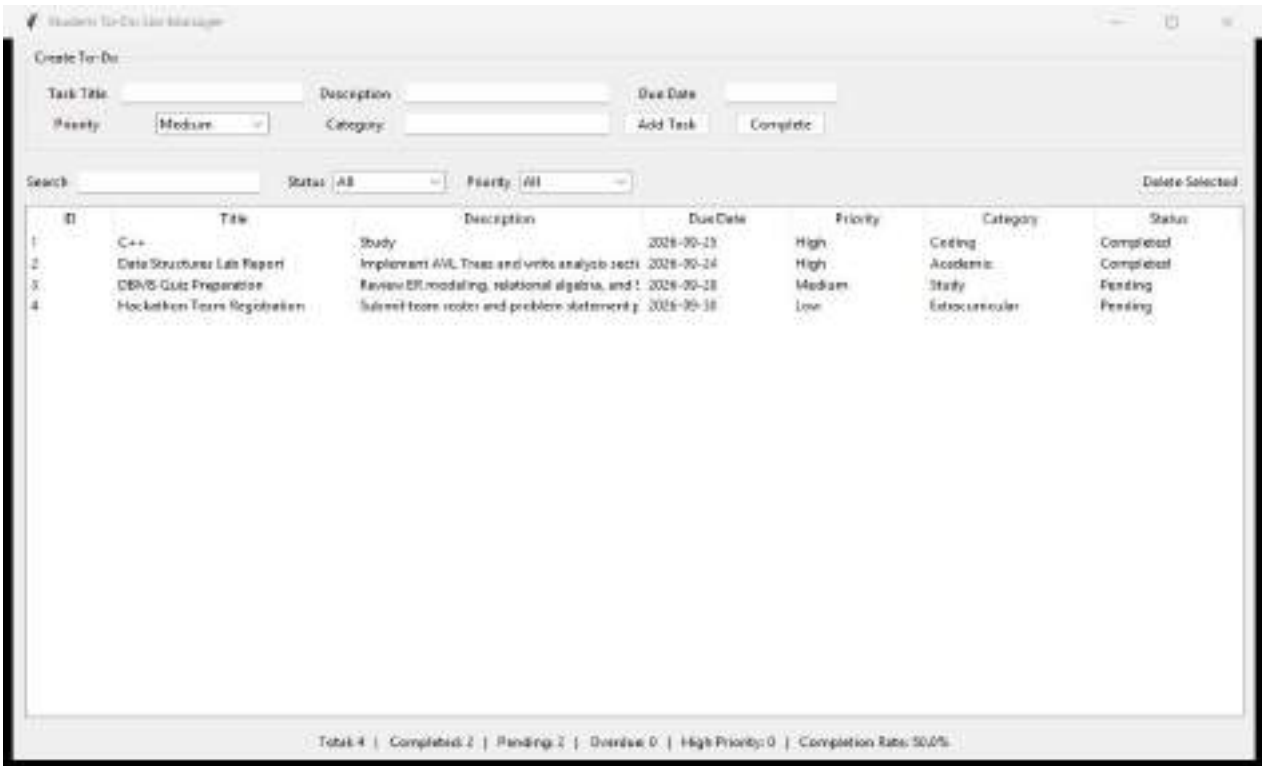


Figure 5: Task #2 marked as 'Completed'. The analytics bar instantly recalculates the completion rate to 50.0%.

12. Testing Approach

Testing was carried out to check whether the main features of the application behaved as expected. The following test cases can be used as a record of testing. The final Pass/Fail status should be filled in only after the corresponding feature has actually been tested.

13. Challenges Faced

While developing the project, I faced a few practical problems. Designing the interface so that it remained simple was one of them. I also had to make sure that task data was saved correctly and was still available after restarting the program.

Other difficulties included validating user input, getting search and filtering to work properly, calculating task statistics, keeping the code organised into separate modules, and finding and fixing errors during development. Managing the project with Git and GitHub also gave me practical experience with version control.

Working through these problems helped me understand Python, GUI development, data storage and the general process of building a software project more clearly.

14. Learnings & Key Takeaways

This project gave me practical experience beyond writing small Python programs. I learned how a complete application can be planned, divided into components, developed and tested.

The main things I learned were Python programming, creating a GUI with Tkinter, storing information in JSON, modular programming, input validation, search and filtering, basic testing, and the use of design diagrams. I also became more familiar with Git and GitHub and understood how a project moves from identifying a problem to implementation and testing.

15. Future Enhancements

The current project can be extended in several ways:

1. Calendar View – Show tasks according to their due dates.
2. Reminder Notifications – Give reminders for upcoming deadlines.
3. CSV/PDF Export – Allow the user to export the task list.
4. User Login – Support separate accounts for different users.
5. Cloud Synchronization – Make tasks accessible from multiple devices.
6. Dark Mode – Add another interface theme.
7. Recurring Tasks – Support daily, weekly or monthly tasks.
8. Task Sorting – Allow sorting by deadline, priority or category.

16. References

- Python Documentation
- Tkinter Documentation
- Git Documentation

- GitHub Documentation
- VITyarthi – Build Your Own Project guidelines
- Official documentation and resources consulted during development

Testing Table

Test Case	Action / Input	Expected Result	Actual Result / Status
TC01	Add a valid task	Task is added successfully	Pass
TC02	Enter task details	Details are stored correctly	Pass
TC03	Complete a task	Task status changes to Completed	Pass
TC04	Delete a task	Selected task is removed	Pass
TC05	Search for a task	Matching task is displayed	Pass
TC06	Filter by priority	Tasks with selected priority are displayed	Pass
TC07	Restart application	Previously saved tasks are loaded	Pass
TC08	Enter invalid data	Validation prevents incorrect input	Pass
TC09	Check analytics	Correct task statistics are displayed	Pass